

# Reviewer Pack — Kashin L1-Flatness of Top Laplacian Eigenvector Bounds Max-C...

Entry 182e2d5f7a37 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 23:25:55 UTC

## Kashin L1-Flatness of Top Laplacian Eigenvector Bounds Max-Cut SoS-2 Gap

Entry ID: 182e2d5f7a37 Verdict: INCONCLUSIVE Recorded: 2026-05-18 23:25:55 UTC

### 1. Conjecture

**Field A** (mathematical branch): Bourgain-Kashin geometric dispersion / discrepancy theory — the L1/L2 flatness deficit of a unit vector in  $\mathbb{R}^n$  as a quantitative obstruction to  $\{\pm 1/\sqrt{n}\}$ -roundability (Kashin 1977 ‘Sections of some finite-dimensional sets and classes of smooth functions’; Bourgain-Tzafriri 1987 restricted invertibility; Kashin-Tzafriri 1994 random subspaces; Schechtman 2006 ‘Two observations regarding embedding subsets of Euclidean spaces in normed spaces’). The functional  $\kappa(G) := 2(1 - \|\varphi\|_1/\sqrt{n})$  uses only L1/L2 ratios on a single eigenvector (no  $F_q$  polynomial extension preserves L1/L2 dispersion under polynomial ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe;  $\kappa$  depends on the GRAPH wiring not on the cut function, shielding from Razborov-Rudich natural proofs. ArXiv search ‘Kashin L1 L2 flatness’ AND (‘Max-Cut’ OR ‘Sum-of-Squares’ OR ‘SoS integrality gap’ OR ‘Delorme-Poljak’) returns 0 direct hits with <5 adjacent papers (all on Kashin random-subspace dispersion in compressed sensing or sign-rank lower bounds via Forster’s method, never on the Max-Cut SoS-2 integrality gap). Distinct from blacklisted Bourgain-Tzafriri sub-column dispersion (random column-subset operator norms on sign matrices), Halász L2 spectrum discrepancy (CDF of singular values), Tusnady 2-box discrepancy (axis-aligned box deviation on clause-polarity clouds), Matousek det-LB (max determinant on submatrix families), and Lee-Yang zeros / Fekete capacity / free-cumulant excess on Laplacian spectra — none uses the L1/L2 flatness deficit of the leading L-eigenvector as a SoS-2 obstruction. **Field B** (complexity object): Degree-2 Sum-of-Squares (Delorme-Poljak) integrality gap for Max-Cut on connected 3-regular graphs  $G$ : relative gap  $\rho(G) := DP(G)/MC(G) - 1$  where  $DP(G) = (n/4) \cdot \lambda_{\max}(L_G)$  is the canonical SoS-2 upper bound (tight on vertex-transitive bipartite  $G$ ) and  $MC(G)$  is the exact Max-Cut value. This is the Goemans-Williamson 0.878 / Schoenebeck-Khot-Vishnoi-Tulsiani / Barak-Steurer SOS\_HIERARCHY regime targeted by the SOS\_DEGREE focus; the canonical stepping stone toward unconditional SoS-degree- $d$  Max-Cut lower bounds via discrepancy-method obstructions on degree-2 pseudoexpectation moment matrices.

#### Statement:

For every connected 3-regular graph  $G = (V, E)$  with  $|V| = n \in [10, 22]$ , let  $\varphi \in \mathbb{R}^V$  be the unit- $\ell^2$  eigenvector of  $L_G$  associated with  $\lambda_{\max}(L_G)$  (canonical: numpy.linalg.eigh last column, sign-fixed by setting the entry of largest absolute value to be positive; when  $\lambda_{\max}$  has multiplicity  $> 1$ , use the column returned by eigh after deterministic seeding of the input matrix). Define the Kashin L<sup>1</sup>-defect  $\kappa(G) := 2 \cdot (1 - \|\varphi\|_1/\sqrt{n}) \in [0, 2 - 2/\sqrt{n}]$ ,  $DP(G) := (n/4) \cdot \lambda_{\max}(L_G)$ , and  $\rho(G) := DP(G)/MC(G) - 1$  where  $MC(G)$  is the exact Max-Cut. The conjecture asserts: for every such  $G$ ,  $\rho(G) \leq 16 \cdot \kappa(G)$ ; equivalently, a single

connected 3-regular  $G$  with  $n \leq 22$  satisfying  $\rho(G) > 16 \cdot \kappa(G)$  falsifies the conjecture.

**Rationale (proposer’s reasoning):**

For  $d$ -regular BIPARTITE  $G$  the top  $L$ -eigenvector is exactly  $\pm 1/\sqrt{n}$  alternating, giving  $\kappa = 0$  and DP-tight sign-rounding ( $\rho = 0$ ), so  $\kappa$  vanishes precisely where SoS-2 is tight. Bourgain-Kashin dispersion theory predicts that the  $L^1/L^2$  flatness deficit of a unit vector quantifies the obstruction to  $\{\pm 1/\sqrt{n}\}$ -roundability — and this is exactly the obstruction that DP-tightness sees: the cut induced by  $\text{sign}(\varphi)$  loses  $(1/4) \cdot (n \cdot \varphi^T L \varphi - \text{sign}(\varphi)^T L \text{sign}(\varphi))$  which scales with the  $\varphi$ -vs- $\text{sign}(\varphi)/\sqrt{n}$  discrepancy. Restricting to 3-regular graphs controls the high-degree pathologies (stars, etc.) that would inflate the constant; on this family the Cauchy-Schwarz slack  $1 - \|\varphi\|_1/\sqrt{n}$  is the canonical Kashin invariant and should track  $\rho$  linearly.

**Taxonomy category:** DISPERSION\_DISCREPANCY (status at proposal time: alive)

## 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 3e4bbebaef4e35ea

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across 120 connected 3-regular graphs ( $n \in \{10, 14, 18, 22\}$ , 30 seeds each,  $\text{seed} = (n, \text{trial\_id})$ ), conjecture is SUPPORTED iff every instance satisfies  $\rho(G) \leq 16 \cdot \kappa(G)$ . FALSIFIED if any single instance yields  $\rho(G) > 16 \cdot \kappa(G)$ . Report  $\sup \rho/\kappa$  over  $\kappa > 0.01$  and max violation gap  $\max(\rho - 16\kappa)$ .

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.96       | SAFE             | SAFE             |
| NATURAL PROOFS | SAFE    | 0.95       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.90       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.97       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - Kashin flatness eigenvector Max-Cut Sum-of-Squares integrality gap - L1 L2 ratio Laplacian eigenvector Delorme-Poljak Max-Cut bound - discrepancy flatness top Laplacian eigenvector SoS degree-2 cubic graph

**Top relevant hits considered:** - [<http://arxiv.org/abs/1107.1630v3>] On the Integrality Gap of the Subtour LP for the 1,2-TSP - [<http://arxiv.org/abs/2404.18142v1>] Revisiting Majumdar-Ghosh spin chain model and Max-cut problem using variational quantum algorithms - [<http://arxiv.org/abs/1909.00238v2>] Principal eigenvectors of general hypergraphs - [<http://arxiv.org/abs/2004.14809v1>] Principal Eigenvector of the Signless Laplacian Matrix - [<http://arxiv.org/abs/2504.13765v1>] Modeling L1 Influence on L2 Pronunciation: An MFCC-Based Framework for Explainable Machine Learning and Pedagogical Feed - [<http://arxiv.org/abs/1806.00072v2>] On graph Laplacian eigenvectors with components in  $\{-1, 0, 1\}$  - [<http://arxiv.org/abs/1301.6896v1>] Laplacians on periodic discrete graphs

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.5s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    def generate_random_3regular_graph(n):
        while True:
            edges = set()
            for _ in range(n * 3 // 2):
                u, v = random.sample(range(n), 2)
                if (u, v) not in edges and (v, u) not in edges:
                    edges.add((u, v))
            adj = [set() for _ in range(n)]
            for u, v in edges:
                adj[u].add(v)
                adj[v].add(u)
            if all(len(neighbors) == 3 for neighbors in adj):
                return adj

    def laplacian_matrix(adj):
        n = len(adj)
        L = [[0] * n for _ in range(n)]
        for i in range(n):
            degree = len(adj[i])
            L[i][i] = degree
            for j in adj[i]:
                L[i][j] = -1
        return L

    def max_cut_exact(adj):
        n = len(adj)
        best_cut_value = 0

        def dfs(node, visited, cut_set):
            visited[node] = True
            cut_set.add(node)
            for neighbor in adj[node]:
                if not visited[neighbor]:
                    dfs(neighbor, visited, cut_set)

        for start_node in range(n):
            visited = [False] * n
            cut_set = set()
            dfs(start_node, visited, cut_set)
            cut_value = sum(len(adj[v]) - len(cut_set.intersection(adj[v])) for v in cut_set) / 2
            best_cut_value = max(best_cut_value, cut_value)

        return best_cut_value
```

```

def spectral_norm(A):
    n = len(A)
    x = [1] * n
    y = [0] * n
    for _ in range(10): # Power iteration method
        y = [sum(A[i][j] * x[j] for j in range(n)) for i in range(n)]
        norm_y = sum(y[i] ** 2 for i in range(n))
        if norm_y == 0:
            return 0
        x = [y[i] / math.sqrt(norm_y) for i in range(n)]
    return max(abs(sum(A[i][j] * x[j] for j in range(n))) for i in range(n))

def sign_rounding_value(adj, eigenvector):
    n = len(eigenvector)
    cut_set = [i if eigenvector[i] > 0 else -1 for i in range(n)]
    return sum(len(adj[v]) - len(set(cut_set[i] for i in adj[v])) for v in set(i for i, x in eigenvector if x != 0))

def kashin_ll_defect(eigenvector):
    n = len(eigenvector)
    return 2 * (1 - sum(abs(x) for x in eigenvector) / math.sqrt(n))

random.seed(seed)
n_values = [10, 14, 18, 22]
results = []
instances_tested = 0
max_violation_gap = 0

for n in n_values:
    for _ in range(30):
        adj = generate_random_3regular_graph(n)
        L = laplacian_matrix(adj)
        lambda_max = spectral_norm(L)
        eigenvector = [x[1] for x in sorted(zip(eigenvector, range(n)), key=lambda x: -abs(x[1]))]
        eigenvector = [eigenvector[i] / math.sqrt(sum(x**2 for x in eigenvector)) for i in range(n)]
        kappa = kashin_ll_defect(eigenvector)
        if kappa == 0:
            continue
        MC = max_cut_exact(adj)
        rho = (n * lambda_max / 4) / MC - 1
        instances_tested += 1
        results.append({
            "metric_name": "rho_over_kappa",
            "metric_value": rho / kappa,
            "instances_tested": instances_tested,
            "conjecture_holds": rho <= 16 * kappa,
            "counterexample": f"n={n}, rho={rho}, kappa={kappa}" if rho > 16 * kappa else ""
        })
    max_violation_gap = max(max_violation_gap, abs(rho - 16 * kappa))

mean_value = sum(result["metric_value"] for result in results) / instances_tested
std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / instances_tested)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "metric_name": "rho_over_kappa",
    "metric_value": mean_value,
    "std_value": std_value,
    "support_fraction": support_fraction
}

```

```

        "instances_tested": instances_tested,
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": max_violation_gap > 0 and f"max violation gap={max_violation_gap}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
                           support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        max_violation_gap = max(abs(result["metric_value"] - 16) for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=max violation gap={max_violation_gap} first_failed={first_failed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown_failure_mode")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b757c7ab.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b757c7ab.py", line 95, in run_trial
    eigenvector = [x[1] for x in sorted(zip(eigenvector, range(n)), key=lambda x: -
abs(x[0]))]

```

UnboundLocalError: cannot access local variable 'eigenvector' where it is not associated with a value

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with an `UnboundLocalError` before any data was produced, so the pre-registered criterion across 120 instances could not be evaluated. | next: Fix the eigenvector assignment bug (ensure eigenvector is computed via `numpy.linalg.eigh` before the sorting step) and rerun the 120-seed sweep.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 266236       |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 5167         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 4574         |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 8191         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14236        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15461        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14327        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17625        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 4775         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 350592 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in `research/audit/182e2d5f7a37.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/182e2d5f7a37.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/182e2d5f7a37.tar.gz` (if generated)